



Deep Learning (CSE641/ECE555)

Quiz-1 (15 Marks) (Duration 60 min)



Name

Roll No.

Beginner: 6 Marks

1. How regression and density estimation models can be used for classification? .5 Marks
Regression uses class scores for classification, and Density estimation uses Baye's law.
2. Assume that the input X to some scalar function $f(\cdot)$ is $n \times m$ matrix. What is the dimensionality of the gradient of f with respect to X ? .5 Marks
Same as X $n \times m$, because each element of this matrix represents the partial derivative of f with respect to the corresponding element of X .
3. Which one is more powerful - a two layer neural network without any activation function or a two layer binary decision tree & Why? .5 Marks
NN. In BDT the decision nodes for depth 2 is 4, and the number of leaf nodes is 8.
4. What are the different types of learning methods in ML/DL? 1 Marks
Lecture 1, slide 11
5. Your binary classification network is $y = \sigma(\text{ReLU}(z))$, where the predicted label of input is chosen to be 1 when $\hat{y} \geq 0.5$ and 0 otherwise. What will happen to this network while training? 1 Marks
All samples will be labelled positive.
6. What happens to the receptive field of a 1D convolutional network as more layers are added? .5 Marks
 - (a) It stays the same
 - (b) It decreases exponentially
 - (c) It increases linearly
 - (d) It increases non-linearly
7. Which of the following is true about dropout? .5 Marks
 - (a) Dropout leads to sparsity in the trained weights
 - (b) At test time, dropout is applied with inverted keep probability
 - (c) The larger the keep probability of a layer, the stronger the regularization of the weights in that layer
 - (d) None of the above
8. Can we remove the bias parameter from the fully-connected layer and the convolutional layer before the batch normalization? 1 Marks
Mathematically Yes, since the mean subtraction step in BN will cancel the bias and BN itself has a bias parameter. However, if BN is applied after the activation function, then not always.
9. According to the Universal Approximation Theorem, which type of function can a sufficiently wide feedforward neural network approximate on a compact subset of $\mathbb{R}^n$.5 Marks
 - (a) Only smooth functions
 - (b) Only polynomial functions
 - (c) Any continuous function
 - (d) Only functions with bounded derivatives

Intermediate: 5 Marks

1. Which of these methods will use less CPU RAM: (a) loading model from individual weight files or (b) sequential model loading with meta device? .5 Mark
individual weight files

2. What are the two broad ways to reduce/avoid overfitting (Hint: Think about function approx. in ERM framework)? .5 Marks

Reducing the space (less functionals, simple DNN with fewer modules/layers); Making the choice of f^* less dependent on data (penalty on coefficients, margin maximization, ensemble methods)

3. Which of the following would you consider to be valid activation functions (elementwise non-linearities) to train a neural net in practice and Why? .5 Mark

- (a) $\phi(x) = -\min(2, x)$
 (b) $\phi(x) = 0.9x^2 + 1$
 (c) $\phi(x) = \begin{cases} \min(x, 0.1x) & |x| \geq 0 \\ \min(x, 0.1x) & |x| < 0 \end{cases}$
 (d) $\phi(x) = \begin{cases} \max(x, 0.1x) & |x| \geq 0 \\ \min(x, 0.1x) & |x| < 0 \end{cases}$

(i), (ii), (iii). is linear functions, therefore quite useless as activations.

4. You are training a deep MLP (100 layers) on a binary classification task, using a sigmoid activation in the final layer and a mixture of tanh and ReLU activations for all other layers. You notice weights to a subset of layers stop updating after the first epoch of training, even though your network has not yet converged. Why is this happening? and explain which among the below options might help. 2 Marks

- (a) Increase the size of your training set
 (b) Switch the ReLU activations with leaky ReLUs everywhere
 (c) Add Batch Normalization before every activation
 (d) Increase the learning rate

(ii), (iii).

Classic vanishing gradient problem. Increasing size of the training set (i) doesn't help as the issue lies with the learning dynamics of the network. Varying the learning rate (iv) might help the network learn faster, but as the problem states the gradients to specific layers almost completely go to zero, so the issue seems to be localized to specific layers. (ii) Solves the problem of dying relus by passing some gradient signal back through all relu layers. (iii) Adding BatchNorm prior to every activation ensures the tanh layers have inputs distributed closer to the linear region of the activation, so the elementwise derivative across the layer evaluates closer to 1.

5. An input of size $[2 \times 3 \times 256 \times 256]$ passed through a 2D convolution layer with output channel (12), kernel size (5,5), dilation (2,2), padding=valid, stride (1,3). Find output shape. 1.5 Marks

(2x12x248x83) Look formula of Height and Width of an image on PyTorch's 2D Convolution Documentation.

Advanced: 4 Marks

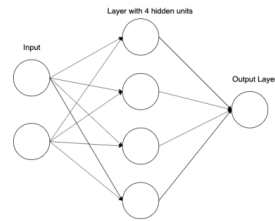
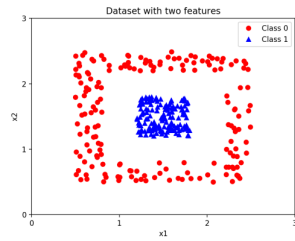
1. After the first DL assignment, your friend Ram concludes that Dropout and Batch Normalization (BN) often lead to a worse performance when they are combined together for CNNs (including ResNets). But your friend Sita argues against it by showing a counter-example of Wide-ResNet (WRN). Is Ram's conclusion wrong? Explain.

Dropout shifts the variance of a specific neural unit when we transfer the state of that network from training to test. However, BN maintains its statistical variance, which is accumulated from the entire learning procedure, in the test phase. The inconsistency of variances in Dropout and BN i.e., the "variance shift") causes the unstable numerical behavior in inference that leads to erroneous predictions finally. Meanwhile, the large feature dimension in WRN further reduces the "variance shift" to bring benefits to the overall performance.

2. You have a dataset where each example contains two features, x_1 and x_2 , and a binary label as shown below. You want to develop a model to perform binary classification using a single hidden layer with 4 neurons. If you use the below mentioned activation function is it possible for this model to achieve perfect accuracy on this dataset? If so, provide a set of weights that achieves perfect accuracy. If not, briefly explain why.

$$\phi(x) = \begin{cases} x \geq 0 : \frac{2x^2}{x+|x|} \\ x \leq 0 : 0 \end{cases}$$

The key is to have each hidden node evaluate one of the sides of the separating square, and the output layer checks that all conditions are true (or false, depending on how the hidden weights are set). For example: $w_1 = (0, -1)$, $b_1 = 2$; $w_2 = (-1, 0)$, $b_2 = 2$; $w_3 = (0, 1)$, $b_3 = -1$; $w_4 = (1, 0)$, $b_4 = -1$; $w_{\text{output}} = (1, 1, 1, 1)$, $b_1 = -4$



Bonus Questions.

1. Derive an expression for the gradient of cross-entropy loss.

1 Point

the cross-entropy loss function for a single example:

$$L(y, \hat{y}) = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Where:

- y is the true label.
- $\hat{y}$ is the predicted logits before applying softmax.
- C is the number of classes.

Apply chainrule

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{\partial L}{\partial o_i} \frac{\partial o_i}{\partial \hat{y}_i}$$

o_i is the raw output (logit) of the i -th class.

$$\begin{aligned} \frac{\partial L}{\partial o_i} &= - \frac{\partial}{\partial o_i} \sum_{j=1}^C y_j \log(\hat{y}_j) \\ &= - \frac{y_i}{\hat{y}_i} \end{aligned}$$

$$\frac{\partial o_i}{\partial \hat{y}_i} = \hat{y}_i(1 - \hat{y}_i)$$

$$\begin{aligned} \frac{\partial L}{\partial \hat{y}_i} &= - \frac{y_i}{\hat{y}_i} \hat{y}_i(1 - \hat{y}_i) \\ &= -y_i(1 - \hat{y}_i) \end{aligned}$$

final gradient w.r.t logits

$$\frac{\partial L}{\partial \hat{y}_i} = \hat{y}_i - y_i$$